Creating a Species Distribution Model Using a U-Net Architecture

Jackson Newell, Jayden Cozort, Reid Beckes, Md Istiak Ahammed

I. Introduction

In ecology and conservation science, Species Distribution Models (SDMs) are widely used to estimate the spatial distribution of species as a function of environmental, climatic, and geographic variables. These models aim to characterize habitat suitability and predict the probability of species presence across continuous geographic space. SDMs play a critical role in biodiversity conservation, climate change impact assessment, invasive species monitoring, and ecological planning. Accurate spatial predictions can inform conservation policies, optimize resource allocation, and improve our understanding of species–environment relationships.

Traditional SDMs have relied primarily on statistical and machine learning regression techniques such as logistic regression, random forests, and maximum entropy models. While these approaches have demonstrated strong predictive performance on structured tabular data, they often treat environmental predictors independently and may not fully exploit the spatial structure inherent in raster-based environmental datasets. With the rapid advancement of Deep Learning architectures, particularly Convolutional Neural Networks (CNNs), new opportunities have emerged to leverage spatial correlations within raster data. CNNs are specifically designed to capture local spatial patterns through convolutional operations which make them one of the best candidates for modeling species distribution.

The goal of this project is to show that CNN architectures can be used to effectively generate SDMs from environmental rasters. Specifically, this study will investigate the effectiveness and feasibility of a UNet architecture in generating a probability distribution to be used as an SDM.

II. Background

A. Image Segmentation

Image segmentation involves the partitioning of an image into multiple different regions to better analyze the image. image segmentation has been used in many applications, from medical imaging to autonomous vehicles and facial recognition. Image segmentation can be split into three types: semantic, instance, and panoptic segmentation. Semantic segmentation labels each pixel with a corresponding class, while instance segmentation extends the segmentation by identifying distinct instances instead. Panoptic is a combination of semantic segmentation and instance segmentation by labeling each pixel with both a class and instance ID. There are multiple different methods to perform image segmentation, these methods are: thresholding, edge-based segmentation, region-based segmentation, clustering-based segmentation, watershed, and deep learning-based segmentation. In this paper, deep learning-based segmentation will be the main method.

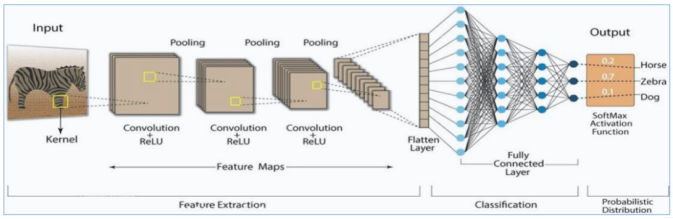

Fig. 1. Overview of CNN process

B. Convolutional Neural Networks

Convolutional Neural Networks(CNN) are a type of deep neural network that uses convolution on grid-like data to capture spatial relationships between data-points. A CNN consists of multiple layers starting with the input layer, which is just the raw data in a grid form. The input layer then passes through the convolutional layers, which are used to extract important features from the input. The layers apply convolution filters on the data, generally with a 3 by 3 kernel, to extract important features. The data is then flattened and fed into the fully connected layer that then performs classification based on the features extracted. This prediction is then sent to the output layer, which is then processed using an activation function.

C. The U-Net Architecture

UNet is a type of Convolutional Neural Network, given its name from its ”U” like appearance. UNet is mainly utilized for image segmentation applications. The architecture consists of three main parts: encoder, bottleneck, and decoder. The encoder uses convolution layers with 3 by 3 kernel filters to scan the image for features. This is then followed by the application of a ReLU activation function which is then proceeded by the use of max pooling with a 2 by 2 kernel to shrink the image whilst keeping important information. In a UNet architecture the encoder is run multiple times until it reaches the bottleneck where the data is in its most compressed form and is then fed into the decoder. The decoder then uses up-sampling to increase the size of the image and counteract the shrinking done by the encoder, it then combines the information from the encoder using skip ”connections” to bring back fine detail that may have been lost from the down-sampling. The decoder then uses convolution with a 2 by 2

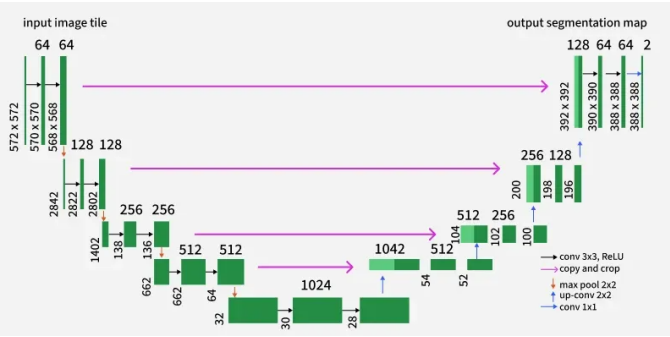

Fig. 2. Image of U-Net architecture

kernel to further refine the output. The number of decoder blocks is equal to the number of encoder blocks.

III. Literature Review

In *A Review of Machine Learning Based Species' Distribution Modeling* [1], the paper reviews current widely used implementations of machine learning in the field. Current methods the paper reviewed consisted of Random Forest (RF), MaxEnt, Support Vector Machine (SVM), and lastly a basic Artificial Neural Network (ANN) model. In addition to reviewing the above models, the paper reviews the improvements that come from hybrid approaches with these approaches having an increase in accuracy compared to non hybrid models. The issue that arises from both categories is its inability to deal with complex SDM's with multiple categories of species reducing the accuracy of prediction, highlighting the advantages of DNN's despite the heavier costs.

In *The Performance and Potential of Deep Learning for Predicting Species Distributions* [2], an MLP and CNN were created and tested against more traditional methods. Those methods being RF and MaxEnt. All four models were also used to create an ensemble [3]. The MLP and CNN were trained with both group separation and all data, the metrics used to evaluate these models were Area under the Receiver Operating Characteristic curve (AUC_ROC), Boyce index, and mean Average Precision (mAP) scored on the test set given as the average (standard deviation) between all species for each group. Results Show that when the MLP and CNN are not trained on the full data set, traditional methods tended to perform better, but, when the models were trained on the full set, the models started performing better than the traditional methods. It is notable that the performance of the deep learning models is hindered by data set bias, which leads to exaggerated results mostly in the CNN.

In *Convolutional neural networks improve species distribution modeling by capturing the spatial structure of the environment* [4]. A CNN model and a Non-CNN model were created and tested. Then, the models were compared to a Boosted Tree(BT) model and an RF model. All models were trained on 97,683 occurrences of 4520 plant species located in France. The group evaluated accuracy, area under the curve (AUC), and true skill statistics (TSS) to check the accuracy of the deep learning models while still having more traditional metrics to better compare to the BT and RF models. Results showed that the CNN model had the best prediction over all other models. Additionally, it was shown that the deep learning models performed better the less frequent the species was with RF and BT showing inverse results. Additional results from the AUC showed that outlier species, those with extremely low occurrence rate, heavily affected the models' results, making those results close to the results of the RF model.

Overall deep learning models have shown to help improve predictive results when it comes to large SDM data sets. Even with the increase in computation and setup cost the outcome shows that deep learning is a good direction to head in for this industry. Especially with more SDMs involving a higher rate of occurrences and more species further solidifying the need for a solution to the complex datasets.

IV. Methodology

A. Model

The first model considered was a ResNet model which would perform the classification. The original intent was to try and make a distributed heat map in this way. However, with the desired output needed, this classification method would not work. Briefly discussing the implementation, ResNet was implemented as a ResNet152 model. This means that the model has a depth of 152. This consists of many blocks of two Convolution layers and then residual connections skipping these layers as well. These skip connections help in preventing the exploding and vanishing gradient problem. This model and the UNet model were created using tensorflow tools in python. In addition, Google Colab was used so that the T4 GPU runtime could be used for more efficient performance. The code for the ResNet model was taken from a tutorial on the website 'GeeksforGeeks' [5]. The tutorial uses repeated blocks of two convolution layers and a residual layer. Then it calculates the number of blocks needed based on the depth. In this case, the depth is 152 and has 25 of these blocks. The number of blocks is based on equation 1 below. The ResNet model is no longer included in the notebook for this project. This is to simplify the running and viewing of the notebook.

$$depth = (block_number * 6) + 2 \tag{1}$$

The UNet model was made using a tutorial on the same website as the ResNet tutorial [6]. The UNet model is the current model in the notebook. The implementation uses functions to define the encoders and decoders of the network. Each encoder and decoder has two 2D convolution layers and a 'ReLU' activation function at the end of each. Then, the encoder has a max pooling layer to reduce the size of the raster being propagated. The decoder, on the other hand, has a 2D convolution transpose layer at the front. This allows the decoder to up sample the raster to increase its size. Then, it uses repeated calls to both to construct the model. The 'bottleneck', which is in between the encoders and decoders, is created by defining two convolution layers with activation layers after each, with both being 'ReLU' activation. The output layer is another convolution layer with a 'sigmoid' activation. Another feature of the model are the skip connections to connect the encoder and corresponding

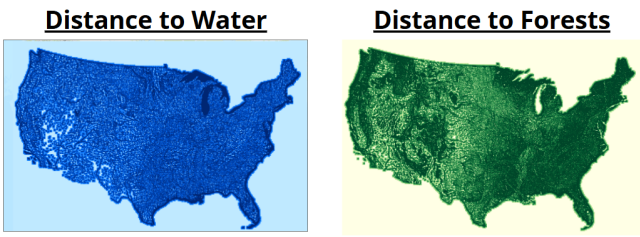

Fig. 3. Distance to Water and Forest Rasters

decoder. These skip connections allow the model to retain some features, so that those features are not lost in down sampling. This model is successful in building and compiling. The model is compiled with an Adam optimizer, a learning rate of 1×10^{-5}, and the binary cross entropy loss function. The input size of the model is (256, 256, 2) and the target size is (256, 256, 1). This target size allows the model to create a probability distribution. The UNet model has about 31 million parameters, all of which can be trained. The full summary of the model is in the notebook in the submitted repository.

B. Data Processing

As mentioned previously, environmental data used in SDMs comes in the form of rasters. A simple SDM for Osprey (*P. h. carolinensis*), a common fish-eating bird, was chosen due to its occurrence throughout the United States. Two environmental factors for Osprey were identified: distance to water and distance to forest [7]. These were chosen after consulting with a Wildlife Ecology student who identified these through literature review. Rasters for these environmental factors were generated from land coverage data from the Multi-Resolution Land Characteristics Consortium (MRLC) [8]. The rasters were generated from the base MRLC raster using euclidean distance. The raster was segmented into blocks (512x512) then combined with its 3x3 block neighborhood to find the distance to either an open water label or forest label. OpenCV was used to threshold and calculate the euclidean distance [9]. Figures 3 shows the generated rasters for distance to water and forests.

Our loss was compared to occurrence data for osprey. Occurrence data is a typical metric to base regression models off of The data was gathered from GBIF in csv format [10]. The data was converted to a raster in ArcGIS using the Feature to Raster tool. The raster has the same pixel size and resolution as the other rasters to maintain spatial consistency. Figure 4 shows the occurrence raster.

V. Experiment Results

The distance to water and forest rasters were combined into a singular 2 channel raster for training. For initial testing, a segment size of 1000 was used with no testing or validation split over 10 epochs. Loss was calculated using Binary Cross Entropy. After training, the training loss and accuracy were measured to test whether the model can effectively train off the data. Results of the test showed that the model could train off the data given. However, the training accuracy showed little improvement as the epochs went on. The loss however is

Fig. 4. Occurrence Raster

decreasing as the epochs go on. This means that overfitting is occurring and that the model would require some fine tuning before a larger test is done. The accuracy and loss graphs are shown below in 5 and 6. The output shown in figure 7 shows a correlation between where there is low distance to water and forests (shown by darker colors) and higher probability for Osprey (shown by lighter colors).

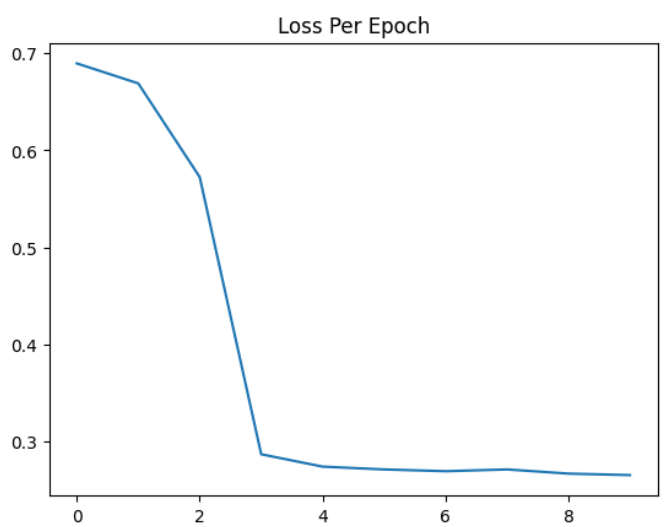

Fig. 5. Loss Graph of UNet Model Training

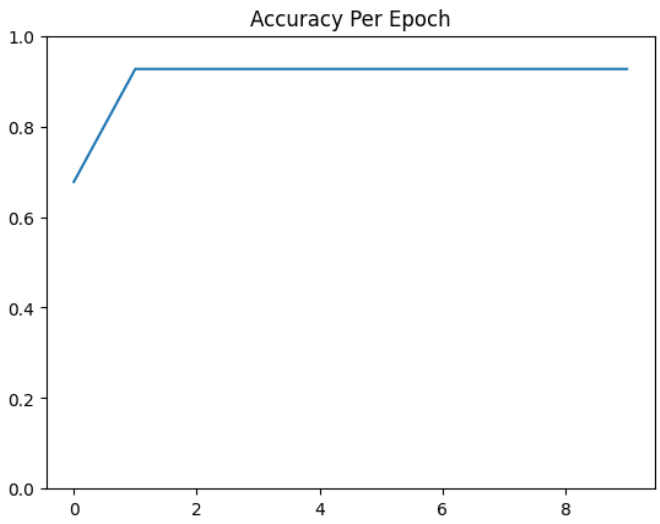

Fig. 6. Accuracy Graph of UNet Model Training

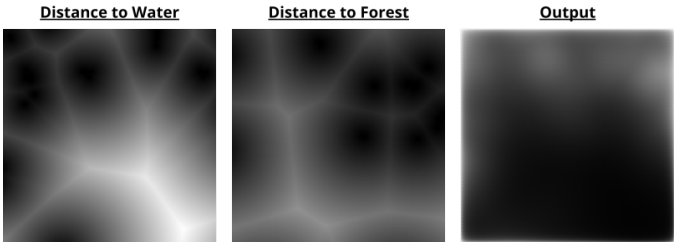

Fig. 7. Example Output from Model

VI. Discussion and Conclusion

The Output given by the model proves that the U-Net architecture can effectively be trained on raster data. This shows that using the image segmentation properties of the U-net can help produce the output given the raster data's pixel-by-pixel nature. With more data inputs such as temperature, humidity, or occurrence data of natural predators, the model could produce a more accurate probability distribution. The extension of the data could be considered as future work for the study. These results show promise for any field that regularly uses raster data as one of its main avenues for information. Most notably in the Environmental science field where this architecture could be manipulated for multiple differing applications, such as migration predictions, habituation endangerment testing, and predicting possible habitat locations for endangered species.

References

[1] J. Zhang and S. Li, "A review of machine learning based species' distribution modelling," in *2017 International Conference on Industrial Informatics - Computing Technology, Intelligent Technology, Industrial Information Integration (ICIICII)*, 2017, pp. 199–206. DOI: 10.1109/ICIICII.2017.76.

[2] B. Kellenberger, K. Winner, and W. Jetz, "The performance and potential of deep learning for predicting species distributions," *Global Ecology and Biogeography*, vol. 35, no. 1, Jan. 2026. DOI: 10.1111/geb.70184.

[3] M. B. Araújo and M. New, "Ensemble forecasting of species distributions," *Trends in ecology & evolution*, vol. 22, no. 1, pp. 42–47, 2007.

[4] B. Deneu, M. Servajean, P. Bonnet, C. Botella, F. Munoz, and A. Joly, "Convolutional neural networks improve species distribution modelling by capturing the spatial structure of the environment," *PLOS Computational Biology*, vol. 17, no. 4, Apr. 2021. DOI: 10.1371/journal.pcbi.1008856.

[5] GeeksforGeeks, *Residual networks (resnet) - deep learning*, Jan. 2026. [Online]. Available: https://www.geeksforgeeks.org/deep-learning/residual-networks-resnet-deep-learning/.

[6] GeeksforGeeks, *U-net architecture explained*, Oct. 2025. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/u-net-architecture-explained/.

[7] M.-L. Bai, D. Schmidt, E. Gottschalk, and M. Mühlenberg, "Distribution pattern of an expanding osprey (pandion haliaetus) population in a changing environment," *Journal of Ornithology*, vol. 150, no. 1, pp. 255–263, Oct. 2008. DOI: 10.1007/s10336-008-0345-3.

[8] Oct. 2024. [Online]. Available: https://doi.org/10.5066/P94UXNTS.

[9] GeeksforGeeks, *Python opencv - distancetransform() function*, Jul. 2025. [Online]. Available: https://www.geeksforgeeks.org/python/python-opencv-distancetransform-function/.

[10] G. B. I. Facility, *Gbif*, 2024. [Online]. Available: https://www.gbif.org/.